

PRÁCTICA N.º 3 — HOJA DE EVIDENCIAS

Construcción y consumo de servicios REST

COM-600 · Microservicios · Gestión 2/2026 · Tema 4 — Comunicación entre Microservicios

Nombre y apellidos	David Franz Soliz Ortega
Código universitario (CU)	35-5261
Carrera y paralelo	Ingeniería de sistemas
Fecha de entrega	03/09/2026
Sistema operativo, versión de Node y de Docker	Windows 10 64 bits, node 24.18.1, Docker 29.7.2
Repositorio (URL)	

Cómo se llena: Este documento se completa mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o del equipo en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Entorno y proyecto base	1, 2, 3	1	
Laboratorio 1 — Diseño del recurso: rutas, verbos y lo que no es REST	4, 5	2	
Laboratorio 2 — CRUD completo con los códigos de estado correctos	6, 7, 8	3	
Laboratorio 3 — Validación de entrada y errores con formato uniforme	9, 10, 11	4	
Laboratorio 4 — Persistencia con MongoDB en contenedor y configuración por entorno	12, 13	5	
Laboratorio 5 — Colecciones grandes: paginación, filtrado y ordenamiento	14, 15	6	
Laboratorio 6 — Versionado, contrato OpenAPI y empaquetado con Docker Compose	16, 17, 18	7, 8	

PARTE 1 — EVIDENCIAS DE LOS LABORATORIOS

Laboratorio 0 — Entorno y proyecto base

Captura N.º 1: salida de los cuatro comandos anteriores en su terminal.

```
PS C:\Users\hp> node --version
v24.18.1
PS C:\Users\hp> npm --version
12.0.2
PS C:\Users\hp> docker --version
Docker version 29.7.2, build a7dcaa6
PS C:\Users\hp> curl.exe --version
curl 8.13.0 (Windows) libcurl/8.13.0 Schannel zlib/1.3.1 WinIDN
Release-Date: 2025-04-02
Protocols: dict file ftp ftps http https imap imaps ipfs ipns mqtt pop3 pop3s smb smbs smtp smtps telnet tftp ws wss
Features: alt-svc AsynchDNS HSTS HTTPS-proxy IDN IPv6 Kerberos Largefile libz NTLM SPNEGO SSL SSPI threadsafe Unicode Unix
Sockets
PS C:\Users\hp> █
```

Pregunta N.º 1. La Práctica 1 trabajó MongoDB desde el shell y la Práctica 2 empaquetó una API con Docker. ¿Qué agrega esta práctica que ninguna de las dos resolvía todavía?

Conexión real: Conecta la API con MongoDB mediante código, logrando que los datos se guarden automáticamente sin usar la consola manual.

Documentación visual: Implementa OpenAPI (Swagger) como un contrato claro para ver, entender y probar la API desde el navegador.

Captura N.º 2: salida de docker ps mostrando el contenedor mongo-practica3 en estado Up.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
NAMES
93ba1d65599f   mongo:7  "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes  27017/tcp
practica3-rest-mongo-1
85365636c8e0   mongo:7  "docker-entrypoint.s..." 8 hours ago   Up 8 hours   0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
mongo-practica3
db45e0e105df   mongo:7  "docker-entrypoint.s..." 9 hours ago   Up 9 hours   27017/tcp
wonderful_bose
PS F:\2-2026\microservicios com600\practicas\practica3-rest> █
```

Captura N.º 3: salida de curl -i http://localhost:3000/salud, donde se lea la línea HTTP/1.1 200 OK y el cuerpo JSON.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i http://localhost:3000/salud
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 19
ETag: W/"13-1gnfvvN/lbzfgVAAnq0r/TYgOnY"
Date: Thu, 03 Sep 2026 19:38:03 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"estado":"arriba"}
PS F:\2-2026\microservicios com600\practicas\practica3-rest> |
```

Laboratorio 1 — Diseño del recurso: rutas, verbos y lo que no es REST

Captura N.º 4: salida de curl -i http://localhost:3000/usuarios mostrando el 200 y los dos usuarios en JSON.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i http://localhost:3000/usuarios
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 134
ETag: W/"86-Gp4Fsms7FgLGZCf+XHxA6jKRtNs"
Date: Thu, 03 Sep 2026 19:39:29 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"nombre":"Maria Flores","correo":"maria@usfx.bo","edad":24},{ "id":2,"nombre":"Juan Perez","correo":"juan@usfx.bo","edad":31}]
PS F:\2-2026\microservicios com600\practicas\practica3-rest> |
```

Captura N.º 5: las dos salidas de curl -i: la ruta correcta con 200 y la ruta inexistente con 404.

200

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i http://localhost:3000/usuarios
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 134
ETag: W/"86-Gp4Fsms7FgL6ZCf+XHxA6jKRtNs"
Date: Thu, 03 Sep 2026 19:08:21 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"nombre":"Maria Flores","correo":"maria@usfx.bo","edad":24},{ "id":2,"nombre":"Juan Perez","correo":"juan@usfx.bo", "edad":31}]
```

404

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i http://localhost:3000/usuario
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Security-Policy: default-src 'none'
X-Content-Type-Options: nosniff
Content-Type: text/html; charset=utf-8
Content-Length: 146
Date: Thu, 03 Sep 2026 19:39:58 GMT
Connection: keep-alive
Keep-Alive: timeout=5

<!DOCTYPE html>
```

Pregunta N.º 2. La guía anterior de la materia usaba rutas como `/crearUsuario` y `/borrarUsuario`. Elija dos de ellas, reescríbalas en estilo REST y explique qué problema concreto evita el cambio para quien consume la API desde otro microservicio.

En el modelo antiguo, el equipo que desarrolla el microservicio consumidor tiene que revisar la documentación para cada acción porque los nombres dependen de la creatividad del programador (`/crearUsuario`, `/nuevoUsuario`, `/addUsuario`, `/insertar_usuario`). En REST, el contrato es uniforme: el recurso es siempre `/usuarios`.

Laboratorio 2 — CRUD completo con los códigos de estado correctos

Captura N.º 6: salida de `curl -i` del POST mostrando HTTP/1.1 201 Created y la cabecera Location con la URL del nuevo usuario.

```
</html>
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d '{ "nombre": "Ana Torres", "correo": "ana@usfx.bo", "edad": 22 }'
>> '{"nombre": "Ana Torres", "correo": "ana@usfx.bo", "edad": 22}'
curl: option -d: requires parameter
curl: try 'curl --help' for more information
{"nombre": "Ana Torres", "correo": "ana@usfx.bo", "edad": 22}
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d '{"nombre": "Ana Torres", "correo": "ana@usfx.bo", "edad": 22}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Security-Policy: default-src 'none'
X-Content-Type-Options: nosniff
Content-Type: text/html; charset=utf-8
Content-Length: 1223
```

Captura N.º 7: las dos salidas de curl -i: el usuario existente con 200 y el inexistente con 404.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i http://localhost:3000/usuarios/1
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 67
ETag: W/"43-cVn4thVFwd6ExICxBf5cnz15aDQ"
Date: Thu, 03 Sep 2026 19:45:56 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":1,"nombre":"Maria Flores","correo":"maria@usfx.bo","edad":24}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i http://localhost:3000/usuarios/999
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 35
ETag: W/"23-w2brRj9rrMQ4PHUxttxSK9tLhwE"
Date: Thu, 03 Sep 2026 19:46:28 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"mensaje":"Usuario no encontrado"}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

Captura N.º 8: salida de curl -i del PUT con 200 y del DELETE con 204 No Content sin cuerpo.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i PUT http://localhost:3000/usuarios
curl: (6) Could not resolve host: PUT
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 134
ETag: W/"86-Gp4Fsms7Fgl6ZCf+XHxA6jKRtNs"
Date: Thu, 03 Sep 2026 19:48:54 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"nombre":"Maria Flores","correo":"maria@usfx.bo","edad":24},{ "id":2,"nombre":"Juan Perez","correo":"juan@usfx.bo","edad":31}]
```

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i DELETE http://localhost:3000/usuarios
curl: (6) Could not resolve host: DELETE
HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 134
ETag: W/"86-Gp4Fsms7Fgl6ZCf+XHxA6jKRtNs"
Date: Thu, 03 Sep 2026 19:49:12 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"id":1,"nombre":"Maria Flores","correo":"maria@usfx.bo","edad":24},{ "id":2,"nombre":"Juan Perez","correo":"juan@usfx.bo","edad":31}]
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

Pregunta N.º 3. Ejecute dos veces el mismo POST y después dos veces el mismo DELETE sobre un usuario existente. Compare los códigos obtenidos y explique, con esos resultados, por qué se dice que DELETE es idempotente y POST no lo es.

En POST cada ejecución altera el estado del servidor generando un nuevo recurso, por lo que repetir la petición acumula efectos por eso no es idempotente y DELETE es idempotente porque el efecto lógico sobre el servidor es el mismo: el recurso ya está borrado. Repetir la operación indefinidamente no cambia el estado final del sistema en

Laboratorio 3 — Validación de entrada y errores con formato uniforme

Captura N.º 9: salida de un POST con edad negativa y correo inválido, mostrando el 400 y el cuerpo de error con los dos detalles.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> $body = '{"nombre":"Juan","correo":"correo-invalido","edad":-5}'
>> curl.exe -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d $body
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-9mMwET9QpAGovRt1LvWvLuLgIw"
Date: Fri, 04 Sep 2026 02:20:28 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":{"codigo":"JSON_INVALIDO","mensaje":"El cuerpo no es JSON válido"}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

Captura N.º 10: las tres respuestas de error (400, 404 y 409) con el mismo formato de cuerpo.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> $body = '{"nombre":"A","correo":"correo-invalido","edad":-5}'
>> curl.exe -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d $body
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-9mMwET9QpAGovRt1LvWvLuLgIw"
Date: Fri, 04 Sep 2026 02:22:49 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":{"codigo":"JSON_INVALIDO","mensaje":"El cuerpo no es JSON válido"}}

PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i "http://localhost:3000/usuarios/999"
HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 84
ETag: W/"54-sQK/5LZ7g6/83Bfi9gpc/P2cHUc"
Date: Fri, 04 Sep 2026 02:23:55 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":{"codigo":"RUTA_NO_ENCONTRADA","mensaje":"Ruta inexistente","detalles":[]}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> $body2 = '{"nombre":"Ana Torres","correo":"ana@usfx.bo","edad":22}'
>> curl.exe -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d $body2
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-9mMwET9QpAGovRt1LvWvLuLgIw"
Date: Fri, 04 Sep 2026 02:28:52 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

Pregunta N.º 4. Un compañero propone responder siempre 200 y poner el resultado real dentro del cuerpo, con un campo transaccion true o false. ¿Qué pierde el consumidor de la API con esa decisión? Mencione al menos dos consecuencias concretas.

Inutiliza las herramientas estándar: Los sistemas de monitoreo, cachés y herramientas de red registrarán todo como "éxito", ocultando los fallos reales porque se guían por los códigos HTTP, no por el JSON.

Mayor complejidad en el código del cliente: El desarrollador ya no puede usar los mecanismos nativos de manejo de errores (como try/catch en Axios o el chequeo de response.ok en Fetch); se verá obligado a procesar y evaluar el cuerpo de cada respuesta manualmente para descubrir si realmente falló.

Captura N.º 11: respuesta 400 con el código JSON_INVALIDO al enviar un cuerpo malformado, en formato JSON y no en HTML.


```
>> curl.exe -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d $body2curl -i -X POST http://localhost:3000/usuarios -H "Content-Type: application/json" -d '{"nombre":'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-9mMwET9QpAGovnrT1LvWvLuLgIw"
Date: Fri, 04 Sep 2026 02:30:07 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":{"codigo":"JSON_INVALIDO","mensaje":"El cuerpo no es JSON válido"}}HTTP/1.1 400 Bad Request
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 77
ETag: W/"4d-9mMwET9QpAGovnrT1LvWvLuLgIw"
Date: Fri, 04 Sep 2026 02:30:07 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":{"codigo":"JSON_INVALIDO","mensaje":"El cuerpo no es JSON válido"}}
```

Laboratorio 4 — Persistencia con MongoDB en contenedor y configuración por entorno

Captura N.º 12: el POST, el reinicio del servidor y el GET posterior mostrando que el usuario sigue existiendo.

```
{"error":{"codigo":"RUTA_NO_ENCONTRADA","mensaje":"Ruta inexista"},"error":{"codigo":"RUTA_NO_ENCONTRADA","mensaje":"Ruta inexista"}
curl.exe -i -X POST http://localhost:3000/v1/usuarios -H "Content-Type: application/json" -d '{"nombre":'
HTTP/1.1 201 Created
X-Powered-By: Express
Location: /usuarios/6a9a2f29e9fe042ca671c870
Content-Type: application/json; charset=utf-8
Content-Length: 89
ETag: W/"59-07KnaCZVJnXvD/Zw8LRyB+8iNM"
Date: Fri, 04 Sep 2026 02:38:33 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -i -X POST http://localhost:3000/v1/usuarios -H "Content-Type: application/json" -d "@datos.json"
HTTP/1.1 409 Conflict
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 88
ETag: W/"58-GPMDb5czTEQRZUONaFa/UAWQJ40"
Date: Fri, 04 Sep 2026 02:39:42 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":{"codigo":"CONFLICTO","mensaje":"El correo ya está registrado","detalles":[]}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

Captura N.º 13: sesión de mongosh mostrando el documento insertado por la API y el índice único sobre correo.

```
0 Bytes / 4.26 GB in use 10 images Last refresh: 2 minutes
Search
Name Tag Image ID Created Size Actions
Terminal
test> use practica3
switched to db practica3
practica3> db.usuarios.find().pretty()

  nombre: 'Ana Torres',
  correo: 'ana@usfx.bo',
  edad: 22
}
]
practica3> db.usuarios.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { correo: 1 }, name: 'correo_1', unique: true }
]
practica3>
```

Pregunta N.º 5. El identificador dejó de ser un número correlativo y pasó a ser el `_id` que genera MongoDB. ¿Qué consecuencia tiene ese cambio para las URL de la API y para el cliente que ya las estaba usando? Relaciónelo con la necesidad de versionar.

Consecuencia: Las URL cambian de números simples a textos largos. Las aplicaciones de los clientes se romperán porque sus códigos esperan procesar números enteros, no cadenas de texto.

Relación con el versionado: Al ser un cambio destructivo, es obligatorio crear una nueva versión de la API (ej. /v1/). Así, los clientes actuales no se quedan sin servicio mientras adaptan su código al nuevo formato.

Laboratorio 5 — Colecciones grandes: paginación, filtrado y ordenamiento

Captura N.º 14: salida de curl mostrando el tiempo y el tamaño de la respuesta sin paginar.

```
+ FullyQualifiedErrorId : CommandNotFoundException

PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -s -o salida.json -w "tiempo: %{time_total}s bytes:
%{size_download}\n" http://localhost:3000/v1/usuarios
tiempo: 0.674269s bytes: 161
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

Pregunta N.º 6. Con el tiempo y el tamaño que acaba de medir, estime qué ocurriría si veinte clientes pidieran la colección completa a la vez. Explique por qué la paginación es una decisión de diseño del servidor y no una comodidad del cliente.

Qué ocurriría con 20 clientes: Si la colección completa es pesada, pedirla 20 veces al mismo tiempo multiplicaría el consumo de RAM del servidor y saturaría el ancho de banda de la red. El servidor se volvería extremadamente lento, se quedaría sin memoria y probablemente colapsaría (se caería). Por qué es decisión del servidor: La paginación (devolver datos de 10 en 10, por ejemplo) es un mecanismo de autodefensa del servidor. Sirve para garantizar que ninguna petición, por más grande que sea, consuma todos los recursos del sistema y deje sin servicio a los demás usuarios.

Captura N.º 15: las tres respuestas: la segunda página, el filtro por edad ordenado y el límite recortado al tope de 100.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> $res = curl.exe -s "http://localhost:3000/v1/usuarios?pagina
=2&limite=5"; $res.Substring(0, [math]::Min($res.Length, 400))
{"datos":[],"paginacion":{"pagina":2,"limite":5,"total":1,"paginas":1}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -s "http://localhost:3000/v1/usuarios?edadMin=60&or
denPor=edad&orden=desc"
{"datos":[],"paginacion":{"pagina":1,"limite":20,"total":0,"paginas":0}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

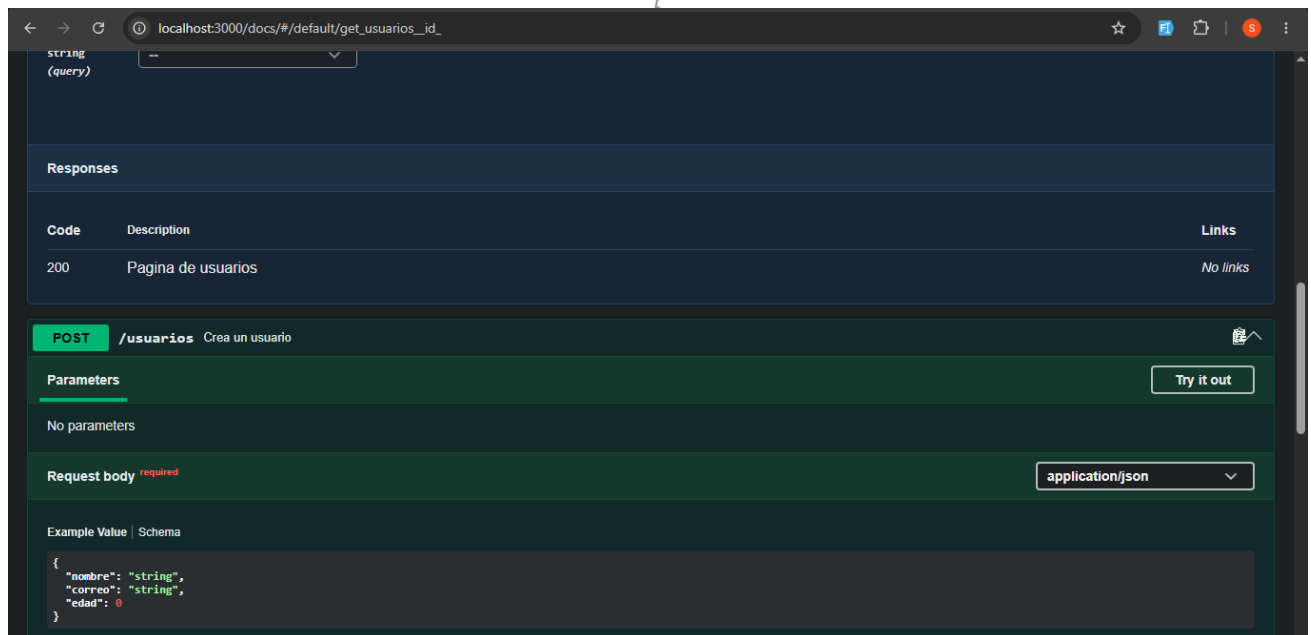
```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> $res = curl.exe -s "http://localhost:3000/v1/usuarios?limite=999999"; $res.Substring(0, [math]::Min($res.Length, 200))
{"datos":[{"_id":"6a9a2f29e9fe042ca671c870","nombre":"Ana Torres","correo":"ana@usfx.bo","edad":22}], "paginacion":{"pagina":1,"limite":100,"total":1,"paginas":1}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

Laboratorio 6 — Versionado, contrato OpenAPI y empaquetado con Docker Compose

Captura N.º 16: las dos respuestas: /v1/usuarios con 200 y /usuarios con el 404 del manejador central.

```
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -s "http://localhost:3000/v1/usuarios"
{"datos":[{"_id":"6a9a2f29e9fe042ca671c870","nombre":"Ana Torres","correo":"ana@usfx.bo","edad":22}], "paginacion":{"pagina":1,"limite":20,"total":1,"paginas":1}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest> curl.exe -s "http://localhost:3000/usuarios"
{"error":{"codigo":"RUTA_NO_ENCONTRADA","mensaje":"Ruta inexistente","detalles":[]}}
PS F:\2-2026\microservicios com600\practicas\practica3-rest>
```

Captura N.º 17: interfaz de OpenAPI en el navegador mostrando el resultado de una petición ejecutada desde ella, con su código de estado.



Pregunta N.º 7. El contrato vive en el repositorio, junto al código. ¿Qué problema aparece cuando el contrato y la implementación se separan, y qué haría usted para que no ocurra en su proyecto integrador?

Problema: Si el contrato (como el archivo openapi.yaml) y el código están en lugares distintos, el principal problema es la desincronización. La documentación dirá una cosa, pero la API real hará otra. Esto rompe el sistema porque los clientes programarán basándose en un contrato desactualizado. Solución en tu proyecto: Para evitarlo, mantendría siempre el archivo de diseño en el mismo repositorio que el código fuente. Además, revisaría que cualquier cambio que modifique las rutas en el código incluya obligatoriamente su respectiva actualización en el archivo

Captura N.º 18: salida de docker compose ps con los dos servicios arriba y la respuesta de curl al servicio contenedorizado.

Pregunta N.º 8. Dentro del compose la variable MONGO_URL apunta a mongodb://mongo:27017 y no a localhost. Explique por qué, y qué ocurriría si dejara localhost en esa variable.

Por qué no usar localhost: En el mundo de Docker, cada contenedor funciona como una computadora independiente. Si dejas localhost en la variable de entorno de tu API, la API buscará la base de datos dentro de su propio contenedor (donde no hay ningún MongoDB instalado). **Por qué usar "mongo":** Al usar mongod://mongo:27017, le estás diciendo a Docker que use su red interna para buscar otro contenedor llamado "mongo" y se conecte a él.

Cierre de la Parte 1

Responda brevemente antes de entregar:

¿Qué parte de la práctica le resultó más difícil y por qué?

Desde la captura 7 y la funcionalidad con Docker porque algunos comandos no funcionaban y hubo que revisar porque fallaba y como resolverlo

¿Qué error tuvo que resolver por su cuenta y cómo lo resolvió?

Algunos comandos de curl no funcionaban correctamente

¿Qué haría distinto si tuviera que repetir esta práctica?

Tener como alternativas el uso de comandos curl.exe e Invoke-WebRequest en caso de que falle curl

Declaración de autoría: Declaro que las capturas presentadas fueron tomadas en mi propio equipo y que las respuestas son de mi autoría. Entiendo que presentar evidencias de otra persona anula la Parte 1 de esta práctica.

Firma del estudiante

Fecha